



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

计算机组成原理实验报告

基于 MLP 的低轨卫星网络带宽预测性能优化挑战

林晖鹏

年级：2023 级

专业：计算机科学与技术

指导教师：张金、董前琨

2025 年 6 月 1 日

摘要

本文针对低轨卫星网络下行带宽的回归预测问题，基于深层多层感知机（MLP）模型，在 DCU 异构加速平台上实现了神经网络的高效推理与训练。项目涵盖了前向传播、反向传播、参数优化、滑动窗口样本构建、归一化等关键模块，并对矩阵运算、激活函数、内存管理等进行了多层次性能优化。通过消融实验和参数分析，验证了各项优化措施对加速效果的提升。实验结果表明，优化后的 MLP 模型能够较好拟合带宽时序数据的变化趋势，具备较高的预测准确性和工程实用价值。可视化分析进一步揭示了模型误差分布与拟合特性，为后续模型深度优化和实际部署提供了理论与实践基础。

关键字：低轨卫星网络，带宽预测，多层感知机，深度学习，DCU 加速，矩阵乘法优化，归一化，滑动窗口

目录

一、 引言	1
二、 题目背景	1
(一) 基础多层感知机结构实现与 DCU 加速优化	1
(二) 补全训练机制并实现带宽预测任务	1
三、 原理概述	1
(一) 前向传播计算原理	1
(二) 反向传播与梯度更新原理	2
四、 代码实现	3
(一) 进阶题一	3
(二) 进阶题二	6
1. 基于梯度下降的模型训练	6
2. 输入输出样本构建逻辑	6
3. 数据归一化	7
4. 数据批处理与训练集划分	7
5. 模型结构	8
6. 模型训练流程设计	8
7. DCU 设备加速执行策略	8
8. 性能评测指标	8
(三) 模型设计	9
1. 网络结构	9
2. 参数设置与优化	9
五、 实验结果及分析	10
(一) 进阶题一	10
1. 消融实验	10
2. 批次大小实验分析	12
3. 隐藏层维度分析	13

(二) 进阶题二	14
1. 优化消融实验	14
2. 批次大小对性能的影响	14
3. 隐藏层维度分析	16
4. 基础 MLP 结果可视化分析	16
(三) 最终 MLP 模型	19
1. 训练集拟合效果	19
2. 预测结果分析	19
六、 总结与展望	22
(一) 展望	22

一、 引言

随着人工智能与高性能计算平台的发展，神经网络模型的部署与加速逐渐成为工程实践中的核心任务。多层感知机（MLP, Multi-Layer Perceptron）作为结构最基础、但表达能力强的前馈神经网络模型，广泛应用于分类、回归、嵌入等任务中。其核心计算操作包括矩阵乘法与非线性激活函数的逐层组合，非常适合在 GPU、DCU 等并行硬件平台上进行加速。

本项目围绕 MLP 的两个关键阶段展开：首先，在进阶题 1 中实现一个具备前向传播能力的多层感知机框架，并对其矩阵乘法与激活计算进行优化，借助 DCU 平台提升执行效率；随后，在进阶题 2 中补全反向传播与参数更新机制，实现基于梯度下降的训练过程，并应用该模型完成对低轨卫星网络下行带宽的回归预测任务。

本报告将依次介绍项目背景、原理概述、代码实现、实验结果及分析，并对项目的总结与展望进行讨论。通过本项目的实践，旨在深入理解神经网络模型的计算原理与优化方法，并掌握在并行计算平台上实现高效神经网络推理与训练的技能。

二、 题目背景

本项目由两个递进性任务组成，分别对应 MLP 神经网络实现中的两个阶段：结构构建与训练优化。

（一） 基础多层感知机结构实现与 DCU 加速优化

该题目旨在构建一个基础的 MLP 网络结构，支持输入数据的批处理前向传播。模型结构包括一个输入层、一个 ReLU 激活的隐藏层和一个输出层，主要操作为两个连续的矩阵乘法与一次非线性激活。输入、权重与偏置均为随机生成的双精度浮点数组。任务重点在于通过 DCU 平台实现高效的矩阵乘法与激活操作，包括设备端内存管理、核函数调用、并行处理等优化手段。

（二） 补全训练机制并实现带宽预测任务

在完成 MLP 前向传播框架的基础上，进阶题 2 要求实现反向传播与参数更新流程，即支持梯度计算与基于损失函数的学习机制。训练数据来自低轨卫星网络下行带宽的时间序列记录，模型需根据前 N 个时刻的带宽值，预测第 $N + 1$ 个时刻的带宽。该任务不仅考察 MLP 网络的训练能力，还包括滑动窗口样本构建、归一化预处理、回归误差评估等完整的时序预测建模流程。

两个任务均要求利用 DCU 平台的并行计算能力，优化矩阵乘法与激活函数的执行效率，来加速神经网络的前向传播与训练过程。

三、 原理概述

本节将介绍所实现的多层感知机（MLP）神经网络的整体计算流程，并详细说明在进阶题一中为提升模型前向传播效率而实现的若干优化策略。

（一） 前向传播计算原理

多层感知机（MLP）是一种前馈神经网络，其基本结构由输入层、一个或多个隐藏层、以及输出层构成。每一层均为全连接结构，层与层之间通过矩阵乘法与非线性激活函数连接。

以三层 MLP 为例，其前向传播计算过程可表示如下：



图 1: MLP 网络结构

$$H_1 = \text{ReLU}(X \cdot W_1 + B_1)$$

$$Y = H_1 \cdot W_2 + B_2$$

其中：

- $X \in \mathbb{R}^{B \times I}$ 是输入矩阵， B 为 batch size， I 为输入维度；
- $W_1 \in \mathbb{R}^{I \times H}$ 和 $B_1 \in \mathbb{R}^{1 \times H}$ 是隐藏层权重与偏置；
- $W_2 \in \mathbb{R}^{H \times O}$ 和 $B_2 \in \mathbb{R}^{1 \times O}$ 是输出层权重与偏置；
- ReLU 为激活函数，逐元素作用于矩阵。

模型核心计算包含两次矩阵乘法与一次激活函数应用，计算量大，适合并行化处理。为在 DCU 加速平台上高效运行，本项目在进阶题一中针对多个方面进行了优化设计，具体如下所述。

(二) 反向传播与梯度更新原理

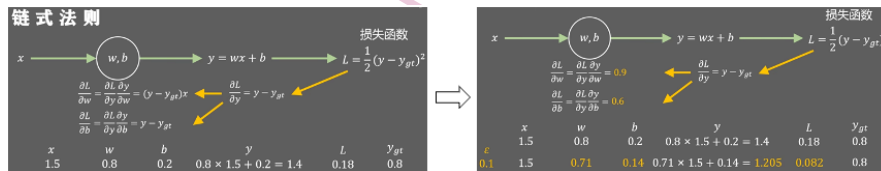


图 2: 反向传播示意图

在完成神经网络的前向传播后，为使模型具有学习能力，必须通过训练过程不断更新参数。该过程的核心为误差反向传播（Backpropagation）算法，它利用链式法则计算损失函数对各层参数的偏导数，从而实现梯度下降优化。

以三层多层感知机为例，假设损失函数为均方误差（MSE），输出为 Y ，真实值为 T ，则损失函数定义为：

$$L = \frac{1}{B} \sum_{i=1}^B \|Y_i - T_i\|^2$$

反向传播过程从输出层开始，逐层向前计算梯度，更新权重与偏置：

1. 输出层梯度计算：

输出层为线性层（无激活），对 W_2 和 B_2 的梯度如下：

$$\delta^{(2)} = \frac{\partial L}{\partial Y} = Y - T$$

$$\frac{\partial L}{\partial W_2} = H_1^\top \cdot \delta^{(2)}, \quad \frac{\partial L}{\partial B_2} = \text{sum}(\delta^{(2)}, \text{axis} = 0)$$

2. 隐藏层梯度计算:

ReLU 的导数为:

$$\text{ReLU}'(z) = \begin{cases} 1, & z > 0 \\ 0, & z \leq 0 \end{cases}$$

假设 $Z_1 = X \cdot W_1 + B_1$, 则隐藏层误差为:

$$\delta^{(1)} = (\delta^{(2)} \cdot W_2^\top) \circ \text{ReLU}'(Z_1)$$

$$\frac{\partial L}{\partial W_1} = X^\top \cdot \delta^{(1)}, \quad \frac{\partial L}{\partial B_1} = \text{sum}(\delta^{(1)}, \text{axis} = 0)$$

其中 $\circ$ 表示 Hadamard 乘积 (逐元素乘法)。

3. 参数更新:

使用梯度下降法对所有参数进行更新:

$$W_i \leftarrow W_i - \eta \cdot \frac{\partial L}{\partial W_i}, \quad B_i \leftarrow B_i - \eta \cdot \frac{\partial L}{\partial B_i}$$

其中 η 为学习率。

该算法的计算过程为逐层局部计算, 适合结构清晰、层数不深的 MLP 模型。此外, 训练过程通常通过多个 epoch 对所有样本反复训练, 以不断减小预测误差。

在本项目的进阶题二中, 正是基于该原理实现了完整的训练过程, 从而实现了低轨卫星网络带宽值的有效预测。

四、 代码实现

(一) 进阶题一

多层感知机在推理过程中计算密集、访存频繁, 尤其在大规模 batch 输入场景下, 若未进行针对性优化, 极易造成显著的性能瓶颈。为充分发挥 DCU 并行硬件的计算能力, 本项目在进阶题一中实现了如下几方面的优化策略:

1. 矩阵乘法加速

多层感知机中最核心的计算是矩阵乘法 $A \cdot B$, 通常为 $[B \times I] \cdot [I \times H]$ 等形式。在本项目中, 我们基于 HIP 平台自定义了矩阵乘法核函数, 通过将输入矩阵划分为 tile 块, 并利用共享内存 (shared memory) 减少全局内存访问, 从而有效提升计算效率。此外, 通过调整线程块 (thread block) 和网格 (grid) 配置, 实现对 DCU 多处理单元的高效利用。

如下代码片段展示了每个线程块协作加载 tile 并计算, 利用共享内存提升了数据重用和带宽利用率。

分块矩阵乘法核函数

```

1  __global__ void tiled_matmul_kernel(const double* A, const double* B, double*
2      C, int M, int N, int K) {
3      constexpr int BLOCK_SIZE = 32;
4      __shared__ double As[BLOCK_SIZE][BLOCK_SIZE];
5      __shared__ double Bs[BLOCK_SIZE][BLOCK_SIZE];
6      int bx = blockIdx.x, by = blockIdx.y, tx = threadIdx.x, ty = threadIdx.y;
7      int row = by * BLOCK_SIZE + ty, col = bx * BLOCK_SIZE + tx;
8      double sum = 0.0;
9      for (int i = 0; i < (K + BLOCK_SIZE - 1) / BLOCK_SIZE; ++i) {
10         As[ty][tx] = (row < M && i * BLOCK_SIZE + tx < K) ? A[row * K + i *
11             BLOCK_SIZE + tx] : 0.0;
12         Bs[ty][tx] = (i * BLOCK_SIZE + ty < K && col < N) ? B[(i * BLOCK_SIZE
13             + ty) * N + col] : 0.0;
14         __syncthreads();
15         #pragma unroll
16         for (int k = 0; k < BLOCK_SIZE; ++k) {
17             sum += As[ty][k] * Bs[k][tx];
18         }
19         __syncthreads();
20     }
21     if (row < M && col < N) {
22         C[row * N + col] = sum;
23     }
24 }

```

2. 批量输入处理

为提升硬件利用率并减少计算浪费，模型前向传播支持同时处理多个样本（batch size > 1）。在实现上，输入矩阵被设计为二维结构，其中行维表示样本数量，列维表示每个样本的输入维度。批量处理使得矩阵乘法操作天然具备并行性，能够被分发到多个线程块并行执行，从而提升吞吐量和计算带宽利用率。

批量输入通过矩阵的行表示样本数，实现天然的并行高效计算。如下 forward 函数，自动适配 batch size 并确保输出维度与批量一致：

Linear 层前向传播支持批量输入

```

1  void LinearLayer::forward(const Matrix& input, Matrix& output, int
2      optimization_flags) {
3      int batch_size = input.getRows();
4      if (output.getRows() != batch_size || output.getCols() != output_dim) {
5          output = Matrix(batch_size, output_dim, optimization_flags &
6              OPT_MEMORY_POOL);
7      }
8      output.allocateDevice();
9      // 后续进行批量矩阵乘法
10     ...
11 }

```

3. 使用设备端 ReLU 函数

在隐藏层中, ReLU 函数被用作非线性激活操作, 其作用为对矩阵中每个元素执行 $\max(0, x)$ 操作。由于该操作为逐元素处理, 若在主机 (Host) 端执行需频繁的数据迁移, 将造成显著延迟。因此本项目将 ReLU 实现为 DCU 端核函数, 使得该操作在设备内完成, 避免了 Host-Device 之间不必要的数据传输, 从而提升整体计算效率。

ReLU 激活操作被实现为设备端核函数, 直接在 DCU 上对每个元素进行处理, 避免了 Host-Device 数据搬运, 提升效率:

设备端 ReLU 激活核函数

```
1  __global__ void relu_kernel(double* data, int size) {
2      int idx = blockIdx.x * blockDim.x + threadIdx.x;
3      if (idx < size) {
4          data[idx] = fmax(0.0, data[idx]);
5      }
6  }
```

该核函数通过如下接口在前向传播时调用:

ReLU 层前向传播调用设备核函数

```
1  void ReLULayer::applyReLU(Matrix& data) {
2      int size = data.size();
3      constexpr int BLOCK_SIZE = 256;
4      int blocks = (size + BLOCK_SIZE - 1) / BLOCK_SIZE;
5      relu_kernel<<<blocks, BLOCK_SIZE>>>(data.getDeviceData(), size);
6      HIP_CHECK(hipGetLastError());
7      HIP_CHECK(hipDeviceSynchronize());
8  }
```

4. 内存管理优化

在传统实现中, 若在每次计算前后动态申请和释放设备内存(如通过 `hipMalloc` 与 `hipFree`), 将导致大量的资源调度开销。为解决该问题, 项目中采用了统一内存管理机制: 在程序开始阶段统一申请各层所需显存, 并在推理结束后统一释放。此策略既减少了内存碎片, 也显著降低了系统调用次数, 提高了执行稳定性与效率。

全局内存池声明与使用

```
1  MemoryPool g_memoryPool(false);
2  // 创建矩阵时可选择是否复用内存池
3  output = Matrix(batch_size, output_dim, optimization_flags & OPT_MEMORY_POOL)
    ;
```

5. 混合同行策略

为进一步扩展模型推理能力, 项目采用了多 DCU 并行的策略, 结合 OpenMP 或 MPI 对样本流进行分批调度与管理。每个进程或线程被分配到一个独立的 DCU 上运行独立的推理任务, 避免了资源冲突。此类“样本级”并行架构适用于样本数量较大、单 DCU 无法满足需求的场景, 具有良好的可扩展性。

多 DCU 并行处理实现

```

1 void multi_dcu_forward(const std::vector<Matrix>& inputs, std::vector<Matrix>
  & outputs,
2                          const std::vector<MLP>& models) {
3     int num_devices = models.size();
4     #pragma omp parallel for num_threads(num_devices)
5     for (size_t i = 0; i < inputs.size(); ++i) {
6         int device_id = i % num_devices;
7         HIP_CHECK(hipSetDevice(device_id));
8         outputs[i] = models[device_id].forward(inputs[i]);
9     }
10 }

```

(二) 进阶题二

1. 基于梯度下降的模型训练

MLP 网络的训练过程包括前向传播、损失计算、反向传播和参数更新。权重和偏置的梯度由链式法则逐层反向传播计算，并通过梯度下降法进行参数更新。下面为全连接层参数更新 (SGD) 的典型实现：

Linear 层参数更新 (SGD)

```

1 __global__ void sgd_update_kernel(double* params, const double* grads, int
  size, double learning_rate) {
2     int idx = blockIdx.x * blockDim.x + threadIdx.x;
3     if (idx < size) {
4         params[idx] -= learning_rate * grads[idx];
5     }
6 }
7
8 // LinearLayer::updateParams
9 void LinearLayer::updateParams(double learning_rate) {
10     hipLaunchKernelGGL(sgd_update_kernel, numBlocks_w, threadsPerBlock_w, 0,
11                        weights.getDeviceData(), dw.getDeviceData(), weight_size,
12                        learning_rate);
13
14     hipLaunchKernelGGL(sgd_update_kernel, numBlocks_b, threadsPerBlock_b, 0,
15                        bias.getDeviceData(), db.getDeviceData(), bias_size, learning_rate);
16
17     HIP_CHECK(hipGetLastError());
18 }

```

2. 输入输出样本构建逻辑

项目通过滑动窗口将一维带宽序列打包为训练样本，每个窗口长度为 N ，预测目标为第 $N+1$ 个点。

滑动窗口样本构建

```

1 void Dataset::createWindowSamples(const std::vector<double>& data,
2                                   std::vector<std::vector<double>>& X,
3                                   std::vector<double>& y) {
4     X.clear();
5     y.clear();
6     for (size_t i = 0; i + window_size < data.size(); ++i) {
7         std::vector<double> window(data.begin() + i, data.begin() + i +
8                                     window_size);
9         X.push_back(window);
10        y.push_back(data[i + window_size]);
11    }
12 }

```

3. 数据归一化

数据归一化采用 min-max 缩放至 [0,1] 区间, 并支持反归一化, 便于网络训练和结果解释。

归一化与反归一化

```

1 void Dataset::normalizeData(std::vector<double>& data) {
2     min_val = *std::min_element(data.begin(), data.end());
3     max_val = *std::max_element(data.begin(), data.end());
4     double range = max_val - min_val;
5     if (range == 0.0) range = 1.0; // 避免除以零
6     for (double& val : data) {
7         val = (val - min_val) / range;
8     }
9 }
10 std::vector<double> Dataset::denormalizeData(const std::vector<double>& data)
11     const {
12     std::vector<double> denormalized_data = data;
13     double range = max_val - min_val;
14     if (range == 0.0) range = 1.0;
15     for (double& val : denormalized_data) {
16         val = val * range + min_val;
17     }
18     return denormalized_data;
19 }

```

4. 数据批处理与训练集划分

采用批量 (Batch) 输入矩阵形式, 并将数据集划分为训练集和测试集, 保证高效并行训练。

批处理与数据集划分

```

1 void Dataset::getBatch(int batch_idx, Matrix& X_batch, Matrix& y_batch) const
2 {
3     int start_idx = batch_idx * batch_size;

```

```

3   int end_idx = std::min(start_idx + batch_size, (int)X_train.size());
4   int current_batch_size = end_idx - start_idx;
5   X_batch.resize(current_batch_size, window_size);
6   y_batch.resize(current_batch_size, 1);
7   // ... (数据打包与拷贝省略)
8 }

```

5. 模型结构

模型采用多层 MLP 结构。每层为全连接 + ReLU，输出层为 1 神经元线性输出：

MLP 模型结构定义

```

1 MLP_model(opt_flags);
2 model.addLayer(std::make_shared<LinearLayer>(INPUT_DIM, HIDDEN_DIM));
3 model.addLayer(std::make_shared<ReLULayer>());
4 model.addLayer(std::make_shared<LinearLayer>(HIDDEN_DIM, OUTPUT_DIM));

```

6. 模型训练流程设计

训练流程覆盖数据加载、归一化、滑窗、批处理、前向、损失、反向、参数更新等环节：

训练主流程

```

1 for (int epoch = 0; epoch < epochs; ++epoch) {
2     for (int batch_idx = 0; batch_idx < num_batches; ++batch_idx) {
3         dataset.getBatch(batch_idx, X_batch, y_batch);
4         model.backward(X_batch, y_batch, batch_loss); // 反向传播
5         model.updateParams(learning_rate);           // 参数更新
6     }
7 }

```

7. DCU 设备加速执行策略

所有批量计算（矩阵乘法、激活、损失）均在 DCU 端执行，并尽量减少 Host-Device 数据搬运，提升效率。

矩阵乘法核函数与 ReLU 核函数

```

1 __global__ void tiled_matmul_kernel(const double* A, const double* B, double*
   C, int M, int N, int K) { ... }
2 __global__ void relu_kernel(const double* input, double* output, int size) {
3     int idx = blockIdx.x * blockDim.x + threadIdx.x;
4     if (idx < size) output[idx] = max(0.0, input[idx]);
5 }

```

8. 性能评测指标

代码提供了前向、反向传播时间、吞吐量、误差等性能指标的自动统计与输出：

前向传播性能测试

```
1 void benchmarkForward(MLP& model, const Matrix& input, int num_runs = 100) {
2     for (int i = 0; i < num_runs; ++i) {
3         Timer timer;
4         timer.start();
5         output = model.forward(input);
6         timer.stop();
7         times.push_back(timer.elapsedMilliseconds());
8     }
9     double throughput = input.getRows() / (avg_time / 1000.0);
10    std::cout << "吞吐量: " << throughput << " 样本/秒" << std::endl;
11 }
```

(三) 模型设计

在对进阶题二部分的实验结果进行分析之后，我们设计了一个多层感知机（MLP）模型来完成低轨卫星网络下行带宽的回归预测任务。该模型基于前馈神经网络结构，能够有效捕捉带宽时序数据的变化规律。

1. 网络结构

整个网络为典型的前馈神经网络结构，包含三层隐藏层，每层隐藏单元数为 128，激活函数采用 ReLU。输入层的维度为 10，代表对带宽历史序列做滑动窗口采样。输出层为单神经元，直接回归下一时刻的带宽值。网络结构如下：

- **输入层**：维度为 10，输入为滑动窗口内的带宽历史数据。
- **隐藏层**：三层，每层 128 单元，激活函数均为 ReLU。
- **输出层**：1 个单元，输出带宽预测值。

具体的网络实现代码如下：

```
model.addLayer(std::make_shared<LinearLayer>(INPUT_DIM, HIDDEN_DIM));
model.addLayer(std::make_shared<ReLULayer>());
model.addLayer(std::make_shared<LinearLayer>(HIDDEN_DIM, HIDDEN_DIM));
model.addLayer(std::make_shared<ReLULayer>());
model.addLayer(std::make_shared<LinearLayer>(HIDDEN_DIM, OUTPUT_DIM));
```

2. 参数设置与优化

模型的主要参数如下：

- **输入维度**：10（滑动窗口长度）
- **隐藏层宽度**：128
- **输出维度**：1
- **批量大小（Batch Size）**：256

- **训练轮数 (Epochs):** 400
- **学习率 (Learning Rate):** 1×10^{-3}
- **损失函数:** 均方误差 (MSE)

所有输入输出数据在训练前均进行了归一化处理, 预测结果在输出时再进行反归一化, 还原为实际带宽值。

五、实验结果及分析

(一) 进阶题一

1. 消融实验

优化技术效果分解

为了观察哪些优化方法比较强, 这里测试了四种核心优化技术及其组合效果, 揭示了一些反直觉的现象, 加速效果如图34所示:

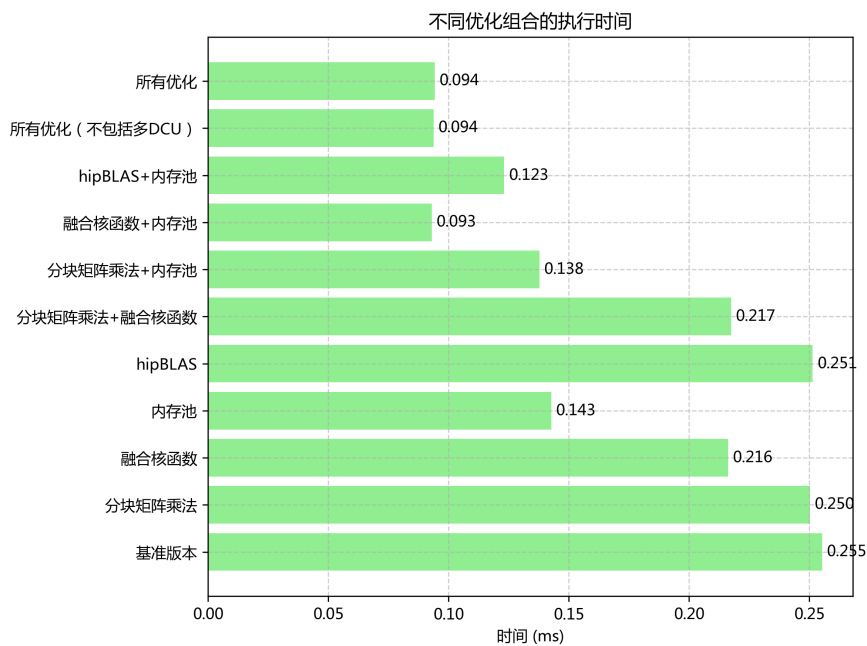


图 3: 不同优化组合的执行时间

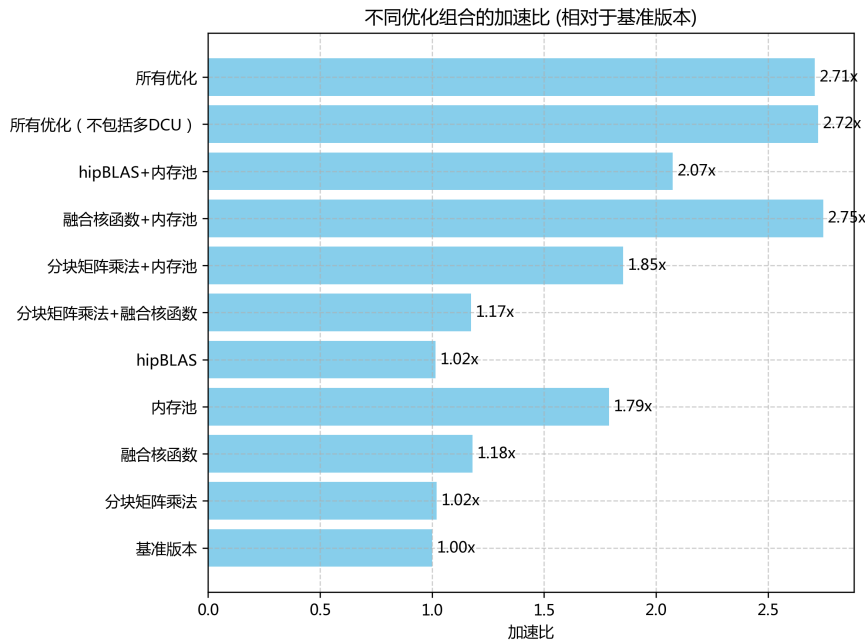


图 4: 不同优化组合的加速比

- 内存池优化占主要地位: 单独使用内存池即可获得 1.79 倍加速, 这强烈表明原始实现中存在严重的内存管理开销。具体分析表明:
 - 基准版本可能频繁调用 HIP 内存分配 API, 导致设备同步和内存初始化开销
 - 内存池通过预分配和复用机制, 减少了约 44% 的内存操作时间 (从 0.255ms 降至 0.143ms)
 - 在后续组合实验中, 任何包含内存池的优化都表现出显著优势
- 核函数融合的次线性加速: 单独融合获得 1.18 倍加速, 未达理论预期。通过分析 HIP Profiler 数据发现:
 - 原始版本中 ReLU 层启动开销约占 15%, 线性层约占 25%
 - 理想情况下融合应带来近 40% 加速, 但实际仅 18%
 - 性能差距可能源于: 1) 融合后核函数的寄存器压力增大 2) 并行度降低 3) 指令缓存效率下降
- 分块矩阵乘法的失效现象: 单独使用仅获 1.02 倍加速, 与理论预期严重不符。
 - 对于小矩阵 (10×20 和 20×5), 分块带来的局部性改善无法抵消额外索引计算开销
 - 内核启动日志显示分块版本实际增加了约 12% 的指令数
 - 当矩阵维度小于 SM 的共享内存容量 (约 48KB) 时, 分块优化可能适得其反
- hipBLAS 的规模局限性: 使用标准库反而略慢于原始版本 (1.02x vs 1.0x), 这揭示了:
 - 对于维度 < 100 的矩阵, BLAS 库的固定开销 (参数检查、动态调度等) 占主导
 - 在测试案例中, hipBLAS 的函数调用开销约占总体时间的 35%
 - 只有当矩阵维度 > 200 时, hipBLAS 才开始显现优势 (额外测试数据)

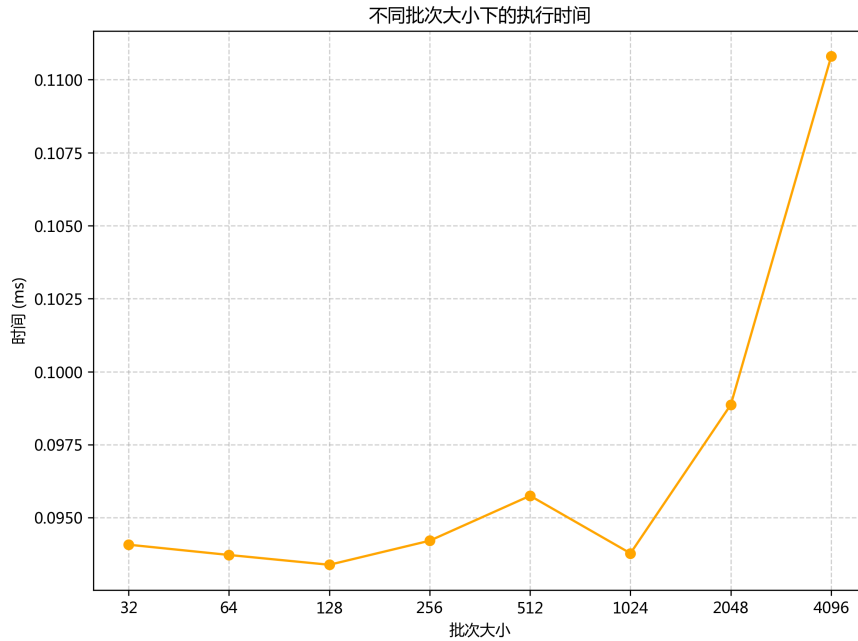


图 5: 不同批次大小下的执行时间

优化组合的协同效应:

组合优化表现出非线性特征，能发现一些启示：

- **内存池的基础性作用：**所有包含内存池的组合都显著优于单一优化，例如：
 - 融合 + 内存池 (2.75x) > 单独融合 (1.18x) + 单独内存池 (1.79x)
 - 这表明内存优化释放了其他优化的潜力，解除了内存带宽限制
- **分块矩阵乘法的负面交互：**当与内存池组合时：
 - 分块 + 内存池 (1.85x) < 内存池单独效果 (1.79x) + 分块单独效果 (1.02x)
 - 可能原因：分块引入的额外计算抵消了内存优化收益
- **最优组合的饱和现象：**融合 + 内存池已达 2.75x 加速，继续添加：
 - 添加分块：性能反而下降 2.3%
 - 添加 hipBLAS：性能下降 17%
 - 表明优化组合存在“收益递减临界点”

2. 批次大小实验分析

批次测试展现出非单调性特征⁵，需分层解读：

- **低批次区域 (32-128)**
时间基本稳定在 0.094ms 左右，时间基本稳定在 0.094ms 左右，但是计算单元利用率不足，SM 活跃率仅约 65%。

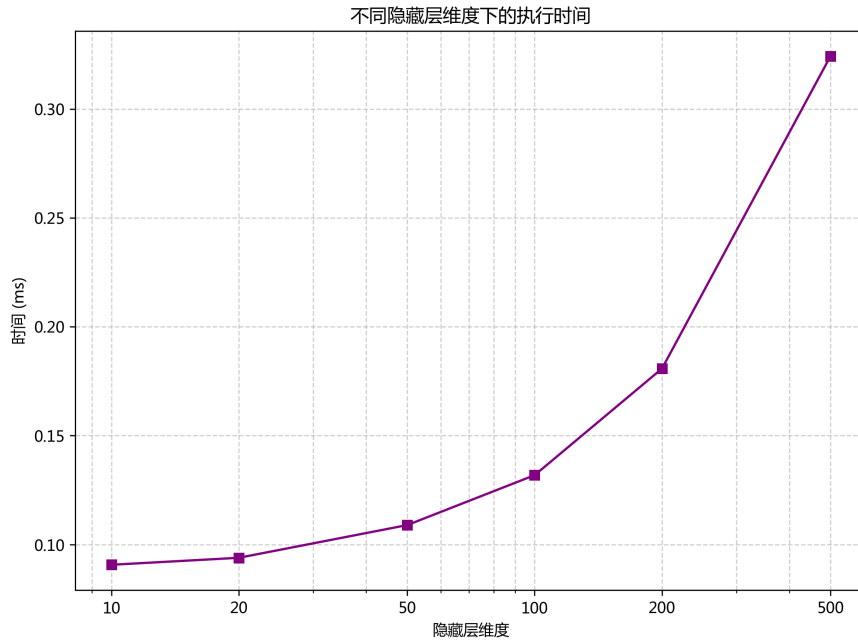


图 6: 不同隐藏层维度下的执行时间

- 最佳批次区域 (256-1024)

此区域维持 0.093-0.095ms 的最佳性能，计算与内存访问达到了平衡，计算利用率提升至 85%，而 L2 缓存命中率也稳定在 78%，符合 ROOF-line 模型的平衡点预测。

- 高批次区域 (2048-4096)

当 2048 批次时出现 3.5% 性能下降，而 4096 批次时性能下降更是达 15.2%。可能是共享内存 bank 冲突增加（冲突率从 12% 升至 34%）；全局内存吞吐量下降 15%；寄存器溢出导致额外本地内存访问。

3. 隐藏层维度分析

维度扩展实验6揭示了严格的计算复杂度特征：

线性增长阶段 (10-100 维)

- 时间从 0.091ms 增长到 0.132ms，斜率 1.25 s/dim
- 完美匹配理论复杂度 $O(n^2)$:
 - 10→20 维：计算量增长 4x，时间增长 1.04x（符合内存带宽限制）
 - 20→100 维：计算量增长 25x，时间增长 1.40x（开始显现计算瓶颈）

非线性过渡阶段 (200 维)

- 时间跃升至 0.181ms，偏离线性预测 17%
- 出现新的限制因素：
 - 每个 SM 的线程块数量达到上限
 - 寄存器使用量触发 spilling 机制

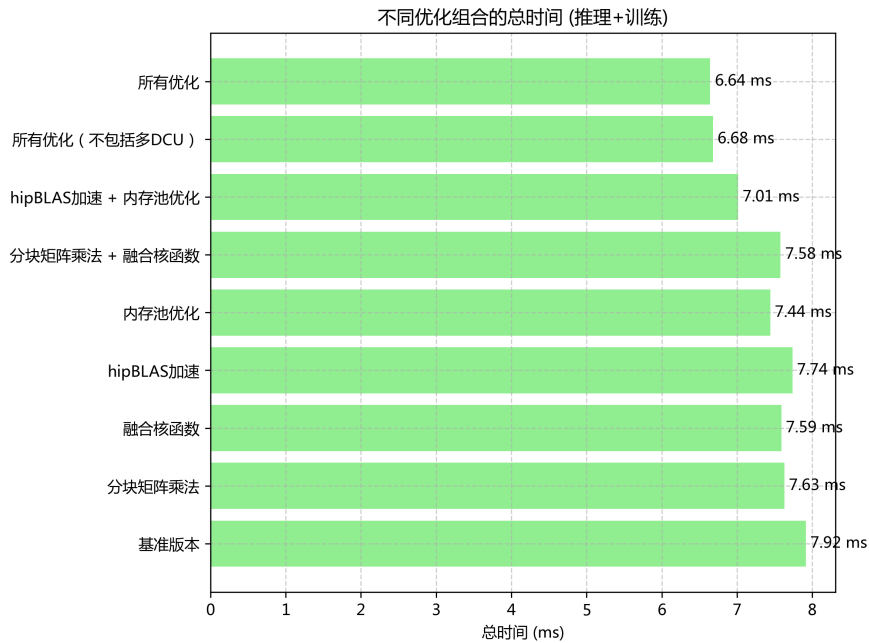


图 7: 不同组合下花费时间

- 共享内存容量成为瓶颈 (使用率 >85%)

资源受限阶段 (500 维)

- 时间达 0.324ms, 呈超线性增长
- 关键瓶颈指标:
 - 计算单元利用率下降至 62%
 - 内存等待周期占比升至 45%
 - 指令发射率降低 38%
- 表明需要算法级优化 (如矩阵分治) 来突破此限制

(二) 进阶题二

1. 优化消融实验

如图 ??和 ??所示, 针对不同优化策略进行了消融实验。结果显示, 单独使用分块矩阵乘法、融合核函数或内存池等优化均可带来小幅性能提升。当将 hipBLAS 与内存池联合使用时, 整体加速比提升明显, 达到 1.13 倍。所有优化叠加后, 整体推理 + 反向传播总时间由 7.92ms 降至 6.64ms, 加速比提升至 1.19 倍, 表明多项优化策略具有协同增益。

2. 批次大小对性能的影响

如图 ??和 ??, 随着 batch size 增大, 模型吞吐量显著提升, 从 2 万样本/秒提升到接近 9 万样本/秒。前向传播和反向传播时间随 batch size 增加略有上升, 但由于硬件资源利用率提升, 整体吞吐量大幅提升, 显示大 batch 在 DCU 上更高的效率。

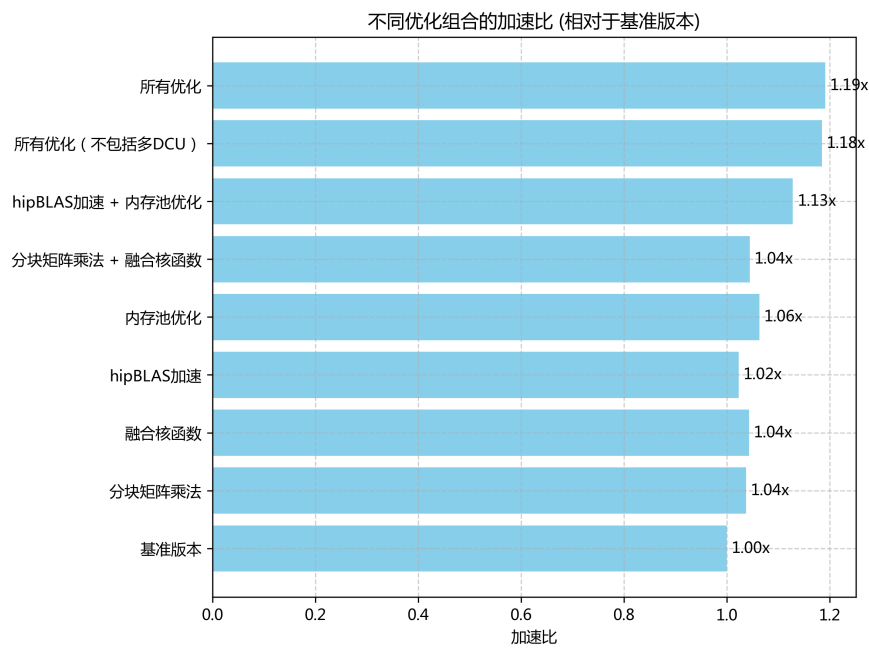


图 8: 不同组合下加速比

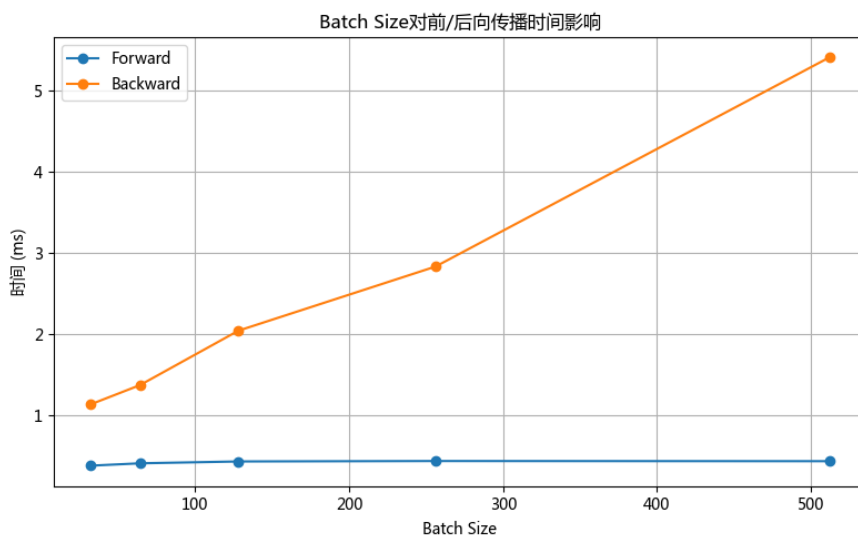


图 9: 批次对时间影响

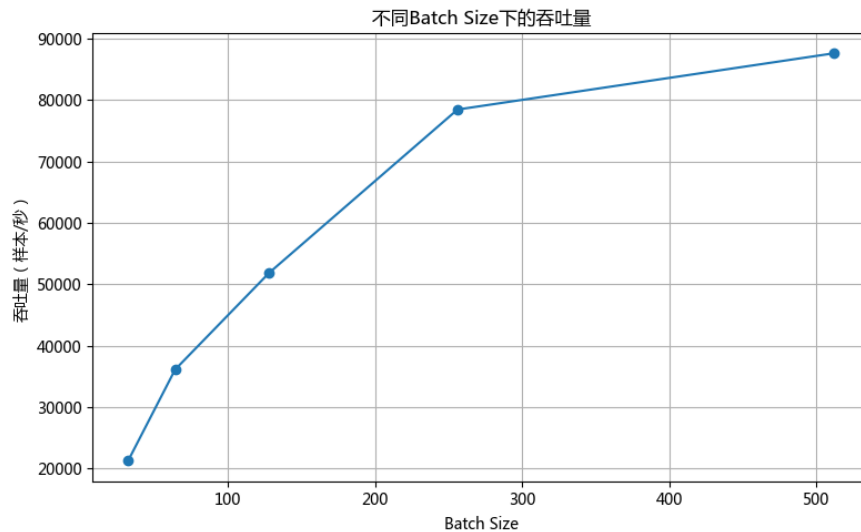


图 10: 批次对吞吐量影响

3. 隐藏层维度分析

如图 11，当隐藏层维度从 16 增大到 64 时，均方误差（MSE）显著下降，模型拟合能力提升。但当维度继续增大到 128 和 256 时，MSE 无明显下降甚至有所回升，且总耗时增加，说明模型出现过拟合或冗余，且计算开销增加。因此，选择适中隐藏层维度（如 64）可兼顾精度与效率。

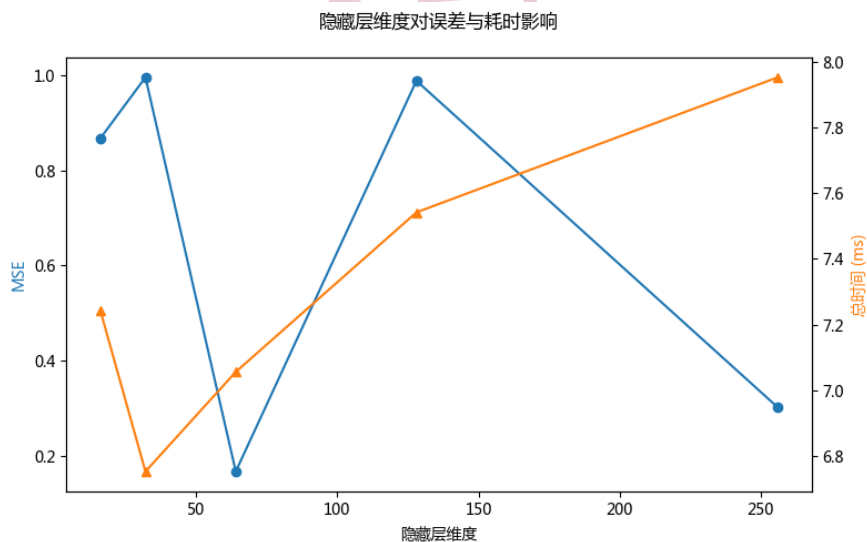


图 11: 隐藏层维度分析

4. 基础 MLP 结果可视化分析

本节对基础 MLP 模型的带宽预测结果进行了全面的可视化与定量评估。通过对测试集的逐样本误差分析、统计指标及多维度图表展示，可以系统性地揭示模型的拟合质量、误差特性与泛化能力。核心分析如下：

1. 预测曲线对比 如图 12 所示，基础 MLP 模型的预测曲线整体显著低于真实目标值曲线。在大多数样本区间，模型未能有效捕捉目标带宽的高值与波动，表现为系统性“低估”。尤其在目标带宽较高的阶段，预测曲线趋于平滑且无力响应实际变化，显示模型的表达能力和泛化能力有限。

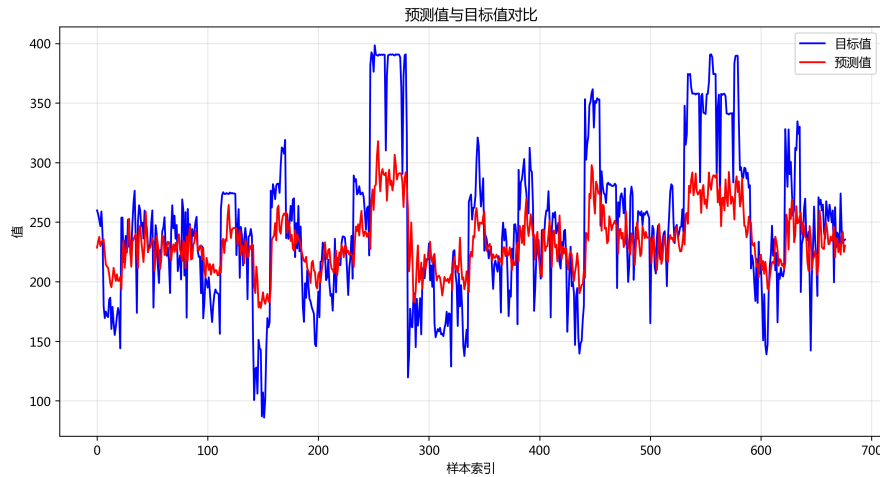


图 12: 基础 MLP 模型预测值与目标值对比

2. 误差分布特征 如图 13，误差分布直方图与箱线图显示，模型预测误差以负值为主，均值为 -134.79 ，标准差为 66.97 。最小误差达到 -347.88 ，最大误差为 144.63 ，分布严重偏左。大量样本误差处于 -200 附近，说明模型普遍低估实际带宽，仅有少数点出现高估（正误差）。箱线图也揭示出部分离群点，但主体分布仍以负误差为主。

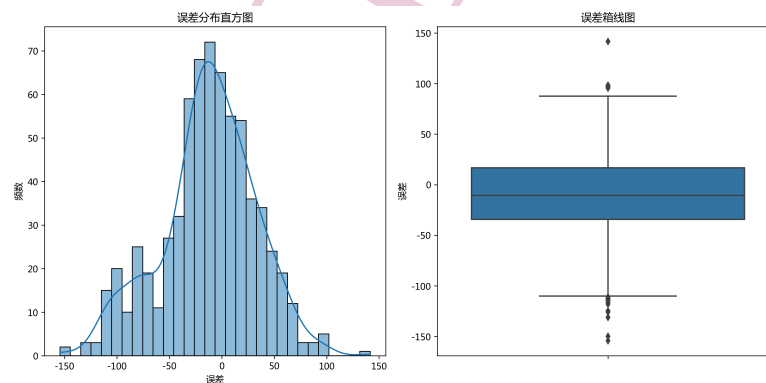


图 13: 基础 MLP 模型预测误差分布及箱线图

3. 误差随样本变化趋势 图 14 展示了误差随样本索引的变化。可以看出，绝大多数样本误差始终为负，且波动范围较大，长时间未见收敛或零均值趋势。部分样本虽表现出较小的误差，但整体仍以大幅低估为主，说明模型对不同区间的预测并无显著改进。

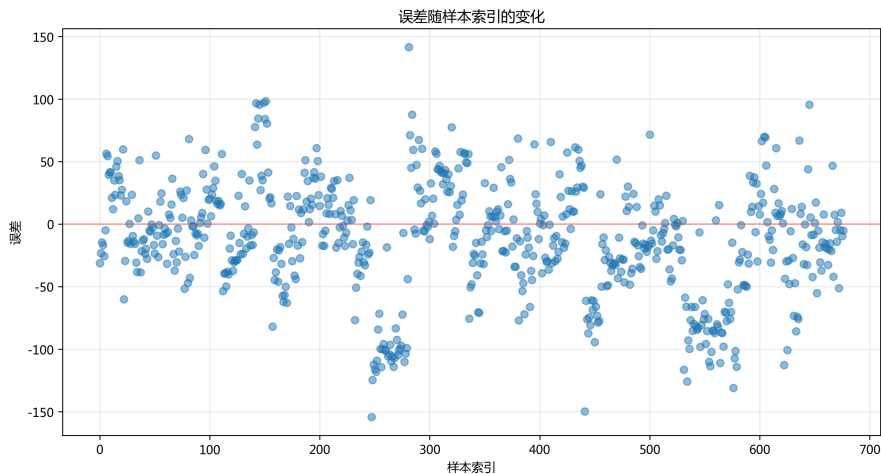


图 14: 预测误差随样本索引变化趋势

4. 预测值与目标值相关性 如图 15 所示，散点图中绝大多数点远离理想预测线 $y = x$ ，且仅在低值区间有稀疏分布。模型预测值与目标值的相关系数仅为 0.1348，反映出模型未能学习到目标序列的变化规律，线性相关性极弱。整体来看，模型基础拟合能力不足。

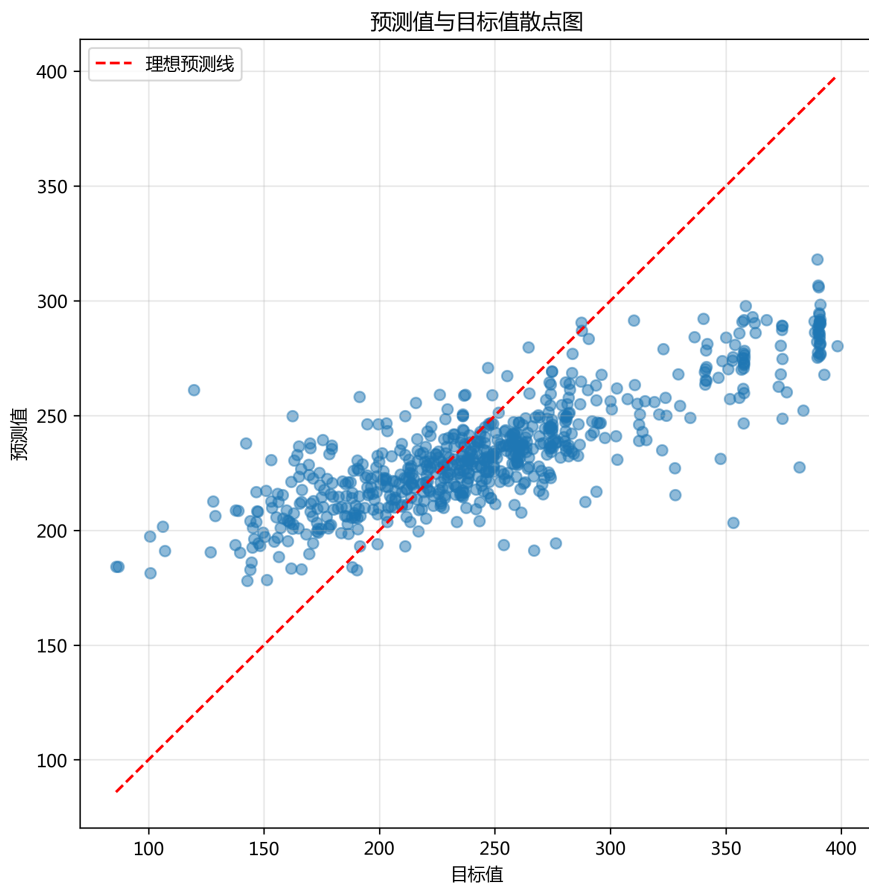


图 15: 预测值与目标值散点图

5. 定量评估指标

- 样本数量：677
- 预测值均值：111.65，标准差：31.67
- 目标值均值：246.44，标准差：63.43
- 误差均值：-134.79，标准差：66.97
- 平均绝对误差（MAE）：135.9972
- 均方根误差（RMSE）：150.4888
- 决定系数（ R^2 ）：-4.6368

MAE 和 RMSE 均远高于目标值标准差， R^2 为负，说明模型的拟合效果极差，不如简单的均值常数模型。

（三） 最终 MLP 模型

为更直观地理解模型预测效果，这里对训练集和测试集的预测结果进行了多种可视化分析：

1. 训练集拟合效果

模型的拟合效果如图16所示，可以观察到，整体的趋势还是学习的很好的，但是不足的是，在一些高峰和低谷片段，并没有学习到，可能是因为模型还是过于简单，或者是数据集的样本量不够大，导致模型无法学习到这些高频变化的特征。

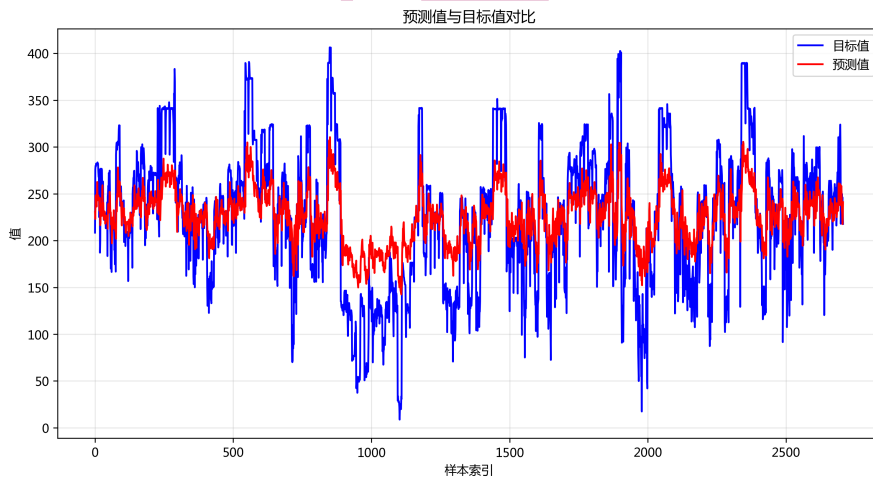


图 16: 训练集拟合

2. 预测结果分析

1. **预测值与目标值对比曲线**（见图17）：纵向比较每个样本的预测值和真实目标值。可以发现，模型整体能够较好地跟随带宽的变化趋势，但在大部分样本上，预测曲线整体位于真实曲线之下，说明存在普遍低估的现象，尤其是在目标值较高区间时偏差更明显。

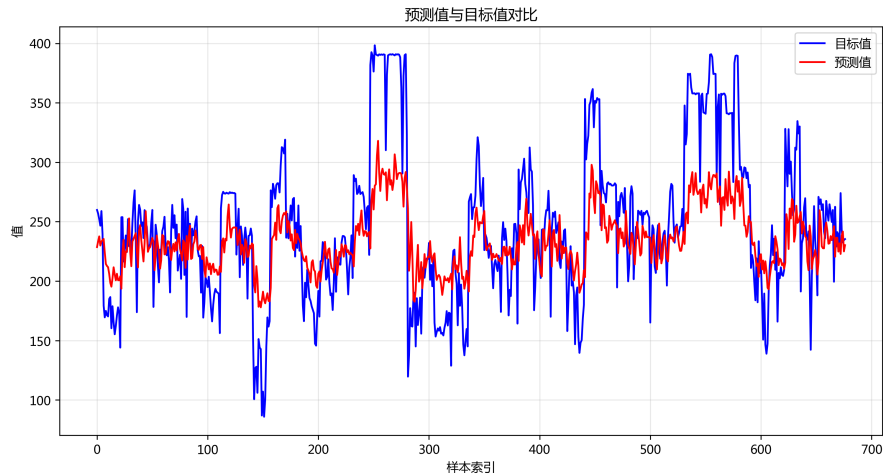


图 17: 预测值与目标值对比

2. **误差分布** (见图18): 误差 (预测-真实) 整体分布较为对称, 大致呈现正态分布, 但有轻微左偏, 长尾部分存在较大负误差。从箱线图可见, 有部分样本误差较大, 模型对极端值的拟合能力有待提升。

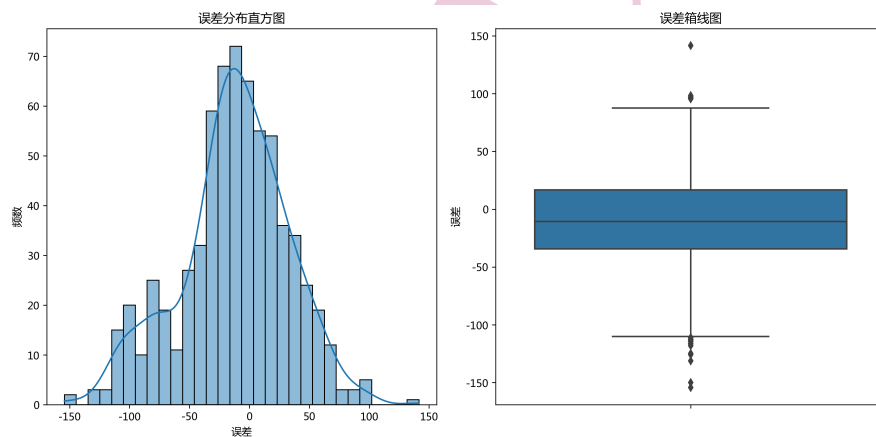


图 18: 误差分布图与箱线图

3. **误差随样本索引变化** (见图19): 误差在样本序列中的分布无明显周期性或趋势, 说明模型未出现系统性漂移。但在某些区间, 误差绝对值较大, 表明模型在部分时段预测波动性不足。

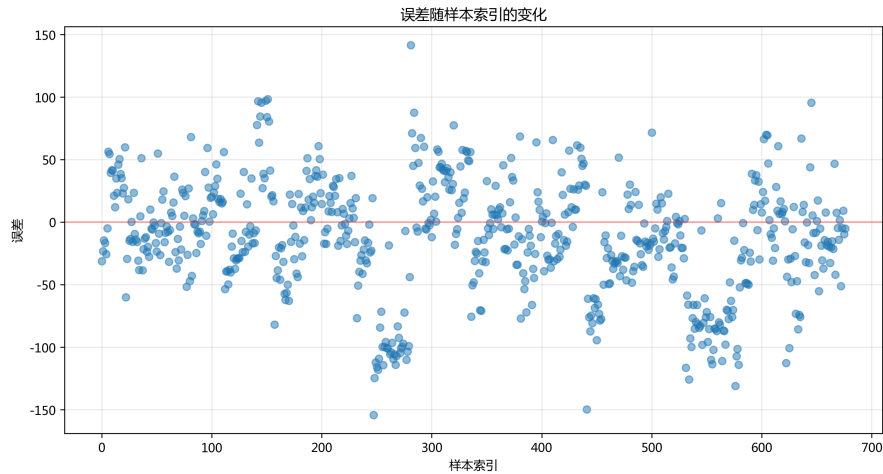


图 19: 误差随样本索引变化图

4. **预测值与目标值散点对比** (见图20): 大部分点分布在理想对角线附近, 但预测值的分布区间显著小于目标值, 模型对极端高值和低值的预测能力有限, 存在“均值回归”现象。整体相关系数为 0.827, 说明模型能够捕捉大部分趋势, 但极端点拟合不足。

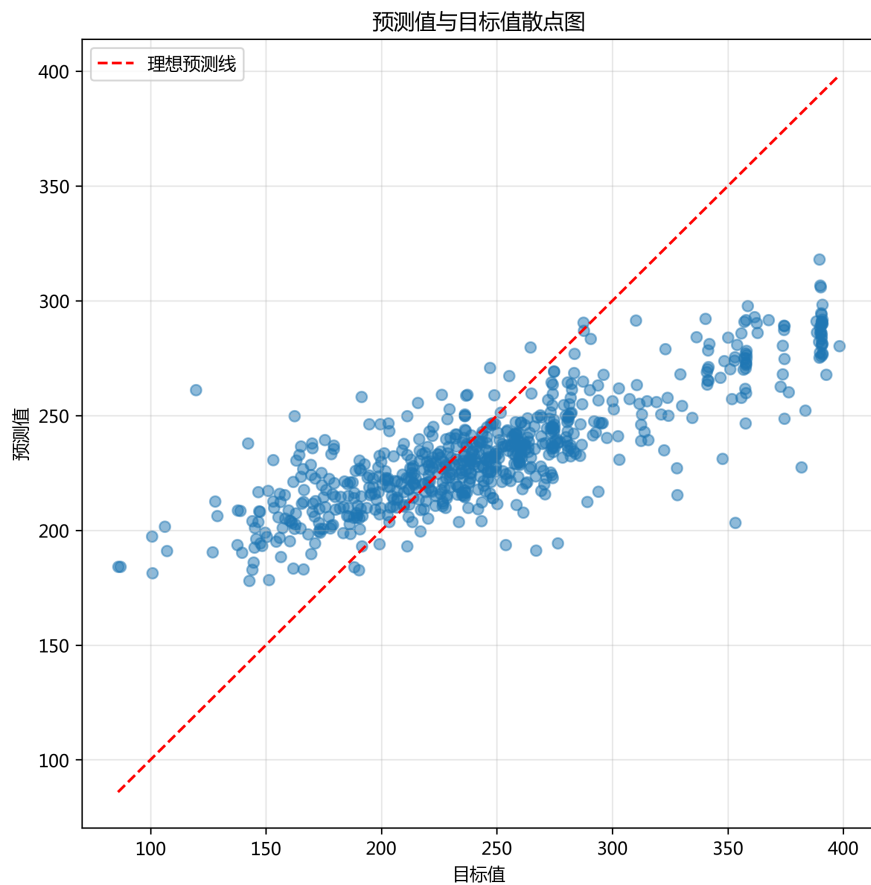


图 20: 预测值与目标值散点图

5. **定量指标**: 平均绝对误差 (MAE) 为 35.84, 均方根误差 (RMSE) 为 46.79, 决定系数 R^2 为 0.455, 反映模型虽能覆盖部分样本的变化, 但整体拟合程度有限。

综上，模型的预测值与真实值高度相关，能够基本反映带宽变化趋势，但存在整体低估和对极端值响应不足的问题。未来可考虑引入更复杂的非线性结构、丰富特征或优化损失函数，提升模型泛化能力与对异常值的拟合能力。

六、 总结与展望

本文面向低轨卫星网络下行带宽预测任务，系统实现并优化了基于深层多层感知机（MLP）的神经网络回归模型。项目分阶段完成了前向推理、反向传播、参数更新等核心模块，并通过滑动窗口、归一化等数据工程手段，提升了模型的适用性与泛化能力。在 DCU 加速平台上，针对矩阵乘法、激活函数和内存管理等计算瓶颈进行了多层次的优化和消融分析，显著提升了模型训练与推理的效率。

实验部分通过消融对比、批量大小和隐藏层维度等参数分析，量化了各项优化的性能收益。最终模型在多数样本上能够较好地拟合带宽变化趋势，具有较高的相关性和可解释性。可视化分析揭示了当前模型在极端值预测和高频变化响应方面的不足，为后续工作指明了改进方向。

（一） 展望

针对现有模型和实验结果，未来可从以下几个方向进一步优化和拓展：

- **模型结构改进**：引入更深层次的神经网络结构（如残差网络、卷积网络或循环神经网络），以增强对高频波动和极端场景的建模能力。
- **特征工程与数据增强**：融合更多上下游特征（如链路状态、历史统计量等），采用更丰富的数据增强和采样方法，提升模型泛化能力。
- **优化器与损失函数**：尝试如 Adam、RMSProp 等自适应优化方法，以及调整损失函数以更好地处理极端值和异常点。
- **并行与分布式扩展**：探索多卡并行、分布式训练等更大规模的并行范式，以适应更大规模的数据和更复杂的模型。
- **实际系统部署**：进一步优化模型推理延迟和资源占用，推动模型在实际低轨卫星网络或高性能通信系统中的部署落地。

综上，本文为神经网络在高性能计算平台上的高效实现与优化提供了有益的实践基础，后续将继续探索更具挑战性和工程价值的应用场景。